Bangladesh University of Engineering and Technology
Department of Computer Science and Engineering

Offline 3 | DFT & FFT

CSE 220 — Signals and Linear Systems

1 Overview

You will write one transform core — a naive $O(N^2)$ DFT and a radix-2 $O(N \log N)$ FFT — and point it at two problems that look nothing alike. **Task A** multiplies integers with tens of thousands of digits by treating their digits as polynomial coefficients. **Task B** blurs a photograph with a large lens-shaped kernel. Both are the same theorem: *convolution in one domain is pointwise multiplication in the other*. Both end with a measured runtime curve of your DFT against your FFT.

1.1 Files

You write these three

- `transforms.py` — `DFTAnalyzer`, `FFTTransformer`, and the optional bonus engine. Both tasks import this one file.
- `bigmul.py` — Task A. `image_conv.py` — Task B.

Each is a skeleton with the required signatures and docstrings. Fill in every `TODO`; do not rename anything.

The two `run_benchmark` functions are also already written: they call your code and produce the required plots, so the timing studies cost you nothing but the time to run them.
Provided, and not to be modified: `io_utils.py` (input files, reproducible random operands, report writing), `image_utils.py` (image load/save, blur kernels, comparison figures) and `bench_utils.py` (timing, log-log plots). These are image handling and plain I/O only — no transform, no convolution and no padding logic lives in them. You also get `inputs/`, `images/` and `expected_outputs/` (what a correct implementation produces; the timings there are from one particular machine).

1.2 Restrictions

- ✗ No `numpy.fft`, `scipy.fft`, `scipy.signal`, `numpy.convolve`, `numpy.correlate`, `scipy.ndimage` or `cv2` — anywhere, including in code you use "just to check".
- ✗ No Python big-integer multiplication of the operands in Task A. Python's integers may be used in exactly one place: verifying your answer at the end.
- ✓ NumPy for ordinary array arithmetic, plus the provided modules and the standard library. Install with `pip install numpy matplotlib pillow`.

1.3 Transform, multiply, transform back

Convolving two sequences of length n costs n^2 multiply-adds, because every sample of one meets every sample of the other. The convolution theorem replaces that with three cheaper steps:

$$a * c \;=\; \mathrm{IDFT}\Big(\mathrm{DFT}\{a\} \cdot \mathrm{DFT}\{c\}\Big),$$

where the multiplication in the middle is *pointwise* — entry k of one spectrum times entry k of the other, N scalar products and nothing more. Two forward transforms, N multiplications, one inverse transform: with the naive DFT that is still $O(N^2)$ and buys you nothing, but with the FFT the whole pipeline costs $O(N \log N)$.

The pipeline, in both tasks

`a, c` ⟶ **pad** ⟶ **FFT** ⟶ `A, C` ⟶ **multiply pointwise** ⟶ `A*C` ⟶ **IFFT** ⟶ $a * c$

Why does convolution become a pointwise product? A convolution is built out of shifts, and complex exponentials are the sequences that shifting leaves alone apart from a constant factor: delaying $e^{2\pi jkn/N}$ by one sample just multiplies it by $e^{-2\pi jk/N}$. The DFT rewrites a signal in that basis. In the time domain every output sample depends on every input sample, so the components are hopelessly tangled; in the frequency basis each component is only ever scaled, never mixed with its neighbours, so the whole convolution decouples into N independent scalar multiplications. The FFT is what makes the change of basis cheap enough to be worth doing.

1.4 The rule both tasks depend on: pad before you multiply

There is one catch, and it is the single place where a correct-looking implementation quietly produces wrong answers.

A length-N DFT knows only N output slots, and the convolution you want is longer than that. Multiplying two length-N spectra pointwise and transforming back gives **circular** convolution with period N, in which anything running past the last slot wraps around and is *added back* onto the start:

$$(a \circledast_N c)[m] \;=\; \sum_{i=0}^{N-1} a[i]\; c[(m-i) \bmod N].$$

circular

What you want is **linear** convolution, where nothing wraps. For inputs of lengths n and q it has $n + q - 1$ entries — longer than either input, which is exactly why the wrap happens if you do not make room:

$$(a * c)[m] \;=\; \sum_{i} a[i]\, c[m - i], \qquad m = 0, 1, \ldots, n + q - 2.$$

The padding rule

Zero-pad both inputs to a common length $N \geq n + q - 1$ before transforming. The extra entries are zeros, so they change none of the arithmetic — they simply give the overflowing coefficients somewhere to land. With the room made, the circular convolution of the padded arrays is exactly the linear convolution of the originals. For the radix-2 FFT, pad further to `next_power_of_two`$(n + q - 1)$.

Example: $a = [1, 2]$, $c = [3, 4]$, so the linear result needs 3 entries. At $N = 2$ you get $[3 + 8,\ 4 + 6] = [11, 10]$ — the x^2 coefficient has wrapped onto the constant term. Padded to $N = 4$ you get $[3, 10, 8, 0]$, the correct $(1 + 2x)(3 + 4x) = 3 + 10x + 8x^2$.

2 Task A: multiplying enormous integers

2.1 Digits are polynomial coefficients

Write an integer in base B as digits $a_0, a_1, \ldots, a_{n-1}$, least significant first, so that $A = \sum_i a_i B^i$. Read that as a polynomial $A(x) = \sum_i a_i x^i$ evaluated at $x = B$. Multiplying two integers is then multiplying two polynomials, and the coefficients of a polynomial product are the linear convolution of the coefficient arrays:

$$p_m \;=\; \sum_{i+j=m} a_i\, c_j \;=\; (a * c)[m].$$

Concretely, take 1234×5678 in base 10, so $a = [4, 3, 2, 1]$ and $c = [8, 7, 6, 5]$ (least significant first). The convolution is

$$p \;=\; [\,32,\ 52,\ 61,\ 60,\ 34,\ 16,\ 5\,],$$

for instance $p_2 = a_0c_2 + a_1c_1 + a_2c_0 = 24 + 21 + 16 = 61$. These are not digits yet — they are far bigger than the base. A **carry sweep** from the least significant end fixes that: 32 leaves digit 2 and carries 3; $52 + 3 = 55$ leaves 5 and carries 5; and so on, giving $[2, 5, 6, 6, 0, 0, 7]$, which read back the other way is 7 006 652. That is the whole algorithm. The only expensive part is the convolution, and Section 1.3 says how to get it cheaply.

Limbs. Nothing forces one decimal digit per coefficient. You may pack several digits into each one: with 4 digits per coefficient the base is $B = 10^4$ and 123456789 becomes $[6789, 2345, 1]$, meaning $1 \cdot B^2 + 2345 \cdot B + 6789$. Each such packed coefficient is called a **limb** — one "digit" of the big number in base B, and one coefficient of the polynomial. Packing is worth doing: a 20 000-digit operand is 20 000 coefficients in base 10 but only 5000 in base 10^4, so the transform is a quarter as long. The skeleton uses $B = 10^4$.

Why not pack more? The mantissa. Your FFT works in double-precision floating point, but the true convolution coefficients are integers, so the last step of `multiply_transform` is to round what comes back. Rounding only recovers the right integer if that integer was representable in the first place. A double carries a 53-bit mantissa, so it holds integers exactly up to $2^{53} \approx 9 \times 10^{15}$ and starts skipping values above it. Each coefficient is a sum of at most n products of two limbs, so

$$p_{\max} \;\le\; n\,(B-1)^2.$$

With $B = 10^4$ a single product is under 10^8, and even a million limbs keeps $p_{\max}$ near 10^{14} — comfortably inside the mantissa, with room to spare for the small rounding error the FFT itself introduces. With $B = 10^9$ the transform would be half as long, but $(B-1)^2 \approx 10^{18}$ exceeds 2^{53} before any summing happens at all: the coefficients could not be represented, and rounding would hand you confidently wrong digits. That is the trade: bigger base, shorter transform, less headroom. Check any change of base against this bound.

Padding, here. The product of an n-limb and a q-limb number has $n+q-1$ coefficients — in the worked example above, four digits times four digits gave seven coefficients. A length-4 transform would have folded p_4, p_5, p_6 back onto p_0, p_1, p_2 and produced a plausible looking, entirely wrong number. Pad to $N \ge n + q - 1$, and to a power of two for the radix-2 FFT.

2.2 What to implement

In `transforms.py`: `next_power_of_two`, `DFTAnalyzer` (`transform`, `inverse`) and `FFTTransformer` (`transform`, `inverse`). The FFT must be radix-2 decimation-in-time — recursive or iterative with a bit-reversal permutation is your choice — must compute the twiddle factors of a stage once per stage, must reuse the same butterflies for the inverse, and must reject a length that is not a power of two. How you write the naive DFT is up to you, as long as it computes the defining sums and is not secretly an FFT.

In `bigmul.py`: `to_limbs`, `from_limbs` (with the carry sweep), `multiply_transform`, `multiply`, `run_single` and `run_benchmark`. Negative operands and zero must work; keep the sign separate from the magnitude so the transform never sees it.

2.3 Running it

Each input file holds two decimal integers, one per line (`#` starts a comment). Provided: `inputs/1.txt` (12 digits, one negative — hand checkable), `2.txt` (200 digits), `3.txt` (2048) and `4.txt` (20000).

```
python3 bigmul.py inputs/1.txt --engine dft --out-dir outputs/task_a/1
python3 bigmul.py inputs/2.txt --engine dft --out-dir outputs/task_a/2
python3 bigmul.py inputs/3.txt --engine fft --out-dir outputs/task_a/3
python3 bigmul.py inputs/4.txt --engine fft --out-dir outputs/task_a/4
python3 bigmul.py --benchmark --out-dir outputs/task_a/benchmark
```

Run inputs 1 and 2 through both engines at some point: identical products are the cheapest check that your FFT agrees with your DFT.

2.4 Outputs

Each single run writes `product.txt` (the product as one decimal string) and `report.txt` (input path, engine, digit counts, base, limb counts, the transform length N used, product length, and the verification verdict). Verification compares against `int(a) * int(b)` and prints `MATCH` or `MISMATCH`. Everything in `report.txt` is written by your code — there is no separate write-up to hand in.

2.5 Benchmark

`run_benchmark` is **already written for you** in the skeleton. It calls your `multiply()` on operands of growing size, drops a curve once one measurement runs past a time budget (so a slow machine gives a shorter curve rather than hanging), and writes `runtime_bigmul.png` with the timing table in `report.txt`. Run it once your transforms work.

What to look at when it finishes: on log-log axes the slope of a curve is its exponent, so the plot should show that the two curves are *not parallel*. Absolute times depend on your implementations, and at small sizes either engine may come out ahead — that is a real effect of constant factors, not a bug. If you implement the optional `multiply_schoolbook`, a third curve appears automatically.

3 Task B: blurring an image in the frequency domain

3.1 A blur is a convolution

Each output pixel of a blur is a weighted average of the input pixels around it, with the weights given by a kernel h:

$$(I * h)[r, c] \;=\; \sum_{i} \sum_{j} I[r - i,\; c - j]\, h[i, j].$$

It is the same operation as Task A with two indices instead of one. Done directly it costs $O(N^2 K^2)$ for an $N \times N$ image and a $K \times K$ kernel: every one of the N^2 output pixels sums K^2 terms.

The kernels. `image_utils.make_kernel` builds three, all normalised to sum to 1 so the picture does not change brightness. A *gaussian* is the familiar soft blur; a *motion* kernel is a straight streak, as if the camera moved during the exposure; and a *bokeh* kernel is a filled disc. The last one is what a real lens does when it is out of focus: a focused lens maps each point of the scene to a point on the sensor, but a defocused one spreads that point into a small image of the aperture — a disc, for a circular aperture. So the kernel *is* that disc, and every bright point in the scene is replaced by a bright disc of the same radius. That is why the night skyline in Figure 1 dissolves into overlapping circles rather than a smooth haze: window lights are near-point sources, and each one becomes a copy of the kernel. It is worth looking at `kernel.png` in your output directory alongside the blurred image.

3.2 The 2D DFT, and why it is two passes of 1D transforms

An image is a two-dimensional signal, so it has a two-dimensional spectrum $F[u, v]$, where u counts vertical oscillations and v horizontal ones:

$$F[u,v] \;=\; \sum_{r=0}^{P-1}\sum_{c=0}^{Q-1} f[r,c]\; e^{-2\pi j\left(\frac{ur}{P}+\frac{vc}{Q}\right)}.$$

Low (u, v) describe slow, broad variation — the sky, a wall; high (u, v) describe fine detail — edges, texture, noise.

Evaluated as written this is hopeless: PQ output entries, each a sum over PQ input pixels, so $O(N^4)$. But the exponential factorises, $e^{-2\pi j(ur/P+vc/Q)} = e^{-2\pi j\, ur/P} \cdot e^{-2\pi j\, vc/Q}$, and only the second factor involves c, so it can be pulled out of the inner sum:

$$F[u,v] \;=\; \sum_{r=0}^{P-1} e^{-2\pi j\, ur/P} \underbrace{\left(\sum_{c=0}^{Q-1} f[r,c]\, e^{-2\pi j\, vc/Q}\right)}_{\text{1D DFT of row } r}.$$

The bracket is just the 1D DFT of row r, and the outer sum is then a 1D DFT down each column of those results. This is **separability**: transform every row, then transform every column of what you got, using the same 1D routine both times. Cost for a square image: $2N$ transforms of length N, so $O(N^3)$ with your naive DFT and $O(N^2 \log N)$ with your FFT — against $O(N^4)$ for the double sum. The inverse works the same way.

The blur then follows Section 1.3 exactly, one dimension at a time: pad, 2D transform of image and kernel, multiply the two spectra pointwise, 2D inverse transform, take the real part, crop. The pointwise multiply is also where the intuition lives — each spatial frequency of the image is simply scaled by the kernel's response at that frequency. A blur kernel has a spectrum close to 1 near the origin and near 0 far from it, so low frequencies pass through untouched while fine detail is multiplied away. Blurring is low-pass filtering, and here you can watch it happen entry by entry.

3.3 What to implement

In `image_conv.py`: `transform_2d` and `inverse_2d` (the row-then-column passes), `convolve_plane` (linear and circular), `convolve_image` (a colour image is three independent planes), `convolve_plane_direct` (the literal four-loop spatial convolution, used as the correctness oracle and as a benchmark curve), `run_single` and `run_benchmark`. The transform core is the same `transforms.py` — you do not write a second one.

3.4 Padding and cropping in two dimensions

The padding rule applies per axis: the full convolution of an (H, W) image with a (k_h, k_w) kernel is $(H+k_h-1,\ W+k_w-1)$, so zero-pad both to at least that, and to the next power of two in each dimension for the radix-2 FFT. Two dimensions add one step: after the inverse transform, take the real part and crop the (H, W) window starting at $(\lfloor k_h/2 \rfloor, \lfloor k_w/2 \rfloor)$. The

kernel sits at the origin of the padded array rather than at its centre, so the result comes out displaced by half a kernel; skipping the crop leaves the whole image shifted diagonally.

You must also produce the same blur *without* padding, transforming at exactly (H, W) with the kernel wrapped around the origin. That is circular convolution, and in two dimensions the wrap is visible: content that should have fallen off one edge of the picture reappears on the opposite edge (Figure 1, right). All provided images are 256×256 or 512×512, so this path works with radix-2 directly.



Figure 1: `images/skyline512.png` with a 19×19 bokeh kernel; each window light becomes a disc. Centre: the correct linear convolution — note that it darkens slightly towards the border, because zero padding treats the world outside the frame as black. Right: the same computation with no padding, which wraps content around from the opposite edge.

3.5 Running it

```
python3 image_conv.py images/skyline512.png --kernel bokeh --param 9 \
    --engine fft --out-dir outputs/task_b/skyline_bokeh
python3 image_conv.py images/sunset512.png --gray --kernel motion --param 41 \
    --engine fft --out-dir outputs/task_b/sunset_motion
python3 image_conv.py images/skyline256.png --gray --kernel gaussian --param 21 \
    --engine dft --out-dir outputs/task_b/skyline256_gaussian_dft
python3 image_conv.py images/skyline512.png --benchmark \
    --out-dir outputs/task_b/benchmark
```

All four are required. The third drives the same pipeline with the naive DFT instead of the FFT, and must produce the same picture.

3.6 Outputs

Each single run writes `blurred.png` (linear), `wraparound.png` (circular), `kernel.png`, `comparison.png` (the three side by side) and `report.txt` (image size, kernel, engine, linear-convolution size, transform size used, verification result).

Verification: convolve the top-left 64×64 corner both ways — through your transform and through `convolve_plane_direct` — and report $\max|\text{spectral} - \text{direct}|$. Expect about 10^{-15}; anything above 10^{-9} is a bug, not rounding. The oracle is never run on a full image because at $O(N^2K^2)$ in pure Python it would take minutes.

3.7 Benchmark

`run_benchmark` is **already written for you** here too. It calls your `convolve_plane` and `convolve_plane_direct` to run two studies on one grayscale plane, and writes both plots and both timing tables:

- **Growing image, fixed kernel** — the DFT route, the FFT route and the direct spatial convolution on $N \times N$ crops. → `runtime_vs_image_size.png`
- **Growing kernel, fixed image** — the direct convolution against the FFT route as the bokeh radius grows. → `runtime_vs_kernel_size.png`

As in Task A, each sweep stops once a measurement exceeds a time budget, so a slow implementation produces a shorter curve instead of hanging.

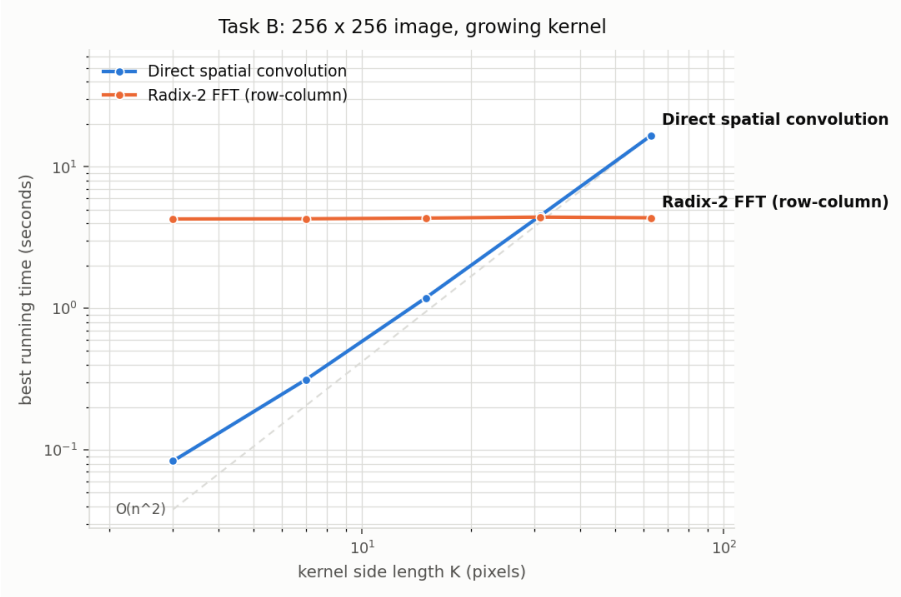


Figure 2: The second study, measured with a correct implementation. The direct cost grows with the kernel; the frequency-domain cost does not depend on the kernel size at all, since the kernel is padded to the same transform size whatever its width. Somewhere in between, the two cross.

4 Bonus

Optional, graded on top of the 100 marks. Each must actually run — leave its output directory in your submission as the evidence.

- **Arbitrary-length FFT (+15).** Implement `ArbitraryLengthFFT` so any N is transformed in $O(N \log N)$: Bluestein's chirp-z algorithm (rewrite the DFT as a convolution and evaluate it with a radix-2 FFT of length $\geq 2N - 1$), or a mixed-radix Cooley-Tukey that factorises N. Then run both tasks with `--engine arbitrary`, transforming at the exact linear-convolution size — in Task B that alone drops the padded area by more than half.

- **Exact multiplication with the NTT (+10).** Replace the complex exponentials of Task A with powers of a primitive root modulo a prime such as $p = 998244353 = 119 \cdot 2^{23} + 1$ (primitive root 3), so every operation is exact and the mantissa argument above disappears entirely. The modulus imposes its own bound in its place — every product coefficient must stay below p, so the base has to come down accordingly. Verify against your floating-point FFT on `inputs/4.txt`.

5 Marks distribution

Component	What it covers	Marks
`transforms.py`	Naive DFT/IDFT and radix-2 FFT/IFFT	25
Task A	Digit packing and carries, spectral multiplication, correct products with verification	33
Task B	Separable 2D transform, linear convolution with padding and cropping, circular demonstration	37
Code	Organisation, readability, no duplicated core	5
Total		**100**
Bonus	Arbitrary-length FFT (Bluestein or mixed-radix)	+15
Bonus	Exact multiplication with the NTT	+10

6 Submission

```
<student_id>/
|-- transforms.py
|-- bigmul.py
|-- image_conv.py
|-- cmd.txt                    # the exact commands you ran
`-- outputs/
    |-- task_a/1..4/           # report.txt and product.txt
    |-- task_a/benchmark/      # runtime_bigmul.png, report.txt
    |-- task_b/<each run>/     # comparison.png, report.txt
    `-- task_b/benchmark/      # both runtime PNGs, report.txt
```

Submit as `<student_id>.zip`. Do not include `inputs/`, `images/`, `expected_outputs/`, the provided modules, virtual environments, cache folders, or the blurred PNGs other than the comparison figures.

Deadline: Saturday, 5 September 2026, 1:00 PM